



PIMPRI CHINCHWAD EDUCATION TRUST'S.
PIMPRI CHINCHWAD COLLEGE OF ENGINEERING
(An Autonomous Institute)

B. TECH

Year: 2026 – 27

Semester: VII

Name of the Student: Swayam Mandhani

PRN: 123B1B184

Department: Computer Engineering

Division: C, **Batch** – C2

Course: DevOps

Course Code: BCE27PE01

Assignment No – 5

Title:

Explore and create a multi-container application consisting of a simple web application and database/service. Configure services, ports, environment variables, volumes, and networking using a compose.yaml file.

Aim:

To develop and run a multi-container application using Docker Compose, consisting of a Flask web application and MongoDB database, while configuring services, ports, environment variables, persistent volumes, and container networking through a compose.yaml file.

Theory:

Docker Compose

Docker Compose is a tool for defining and running multi-container applications using a YAML configuration file. It allows multiple services, networks, volumes, ports, environment variables, health checks, and service dependencies to be configured in one place.

Multi-Container Architecture

The application uses two services. The web service runs a Flask application and is exposed on port 5000. The mongodb service runs MongoDB and is exposed on port 27017. Both services communicate through a dedicated Docker bridge network.

Environment Variables

Environment variables provide configuration to the Flask container without hard-coding the database

connection settings. The application uses MONGO_URI, MONGO_DB, and MONGO_COLLECTION.

Volumes

A named Docker volume, assignment-5_mongodb-data, is mounted at /data/db in the MongoDB container. This allows database data to persist when the MongoDB container is restarted or recreated.

Networking

The services are connected to the Compose-created bridge network assignment-5_assignment5-network. The Flask application reaches MongoDB using the service name mongodb rather than localhost.

Code / Implementation:

Project Structure

Assignment-5/

```
|— app/
|   |— app.py
|   |— requirements.txt
|   |— Dockerfile
|   |— .dockerignore
|— compose.yaml
|— README.md
```

compose.yaml

services:

web:

build:

context: ./app

dockerfile: Dockerfile

container_name: assignment5-web

ports:

- "5000:5000"

environment:

MONGO_URI: mongodb://mongodb:27017/assignment5

MONGO_DB: assignment5

MONGO_COLLECTION: messages

depends_on:

mongodb:

condition: service_healthy

networks:

- assignment5-network

mongodb:

image: mongo:7

container_name: assignment5-mongodb

restart: unless-stopped

ports:

- "27017:27017"

environment:

MONGO_INITDB_DATABASE: assignment5

volumes:

- mongodb-data:/data/db

healthcheck:

test: ["CMD", "mongosh", "--quiet", "--eval", "db.adminCommand('ping').ok"]

interval: 5s

timeout: 5s

retries: 10

start_period: 10s

networks:

- assignment5-network

volumes:

mongodb-data:

networks:

assignment5-network:

driver: bridge

Flask Application

```
MONGO_URI = os.getenv("MONGO_URI", "mongodb://localhost:27017/assignment5")
```

```
MONGO_DB = os.getenv("MONGO_DB", "assignment5")
```

```
MONGO_COLLECTION = os.getenv("MONGO_COLLECTION", "messages")
```

```
client = MongoClient(MONGO_URI, serverSelectionTimeoutMS=3000)
```

```
@app.route("/health")
```

```
def health():
```

```
    client.admin.command("ping")
```

```
    return jsonify({
```

```
        "status": "healthy",
```

```
        "web_service": "running",
```

```
        "database_service": "connected"
```

```
    })
```

```
@app.route("/data")
```

```
def data():
```

```
    document = {
```

```
        "message": "Hello from Flask to MongoDB!",
```

```
        "created_at": datetime.now(timezone.utc).isoformat()
```

```
    }
```

```
    result = collection.insert_one(document)
```

```
    return jsonify({"status": "success",
```

```
                    "inserted_id": str(result.inserted_id)})
```

Implementation Commands

docker compose config

docker compose build

```
docker compose up -d
docker compose ps
docker compose logs --tail=50
docker network ls
docker network inspect assignment-5_assignment5-network
docker volume ls
docker volume inspect assignment-5_mongodb-data
docker compose exec mongodb mongosh
docker compose restart mongodb
docker compose down
```

Output

The application was successfully built and executed using Docker Compose. The Flask web service and MongoDB service ran as separate containers, communicated over a dedicated bridge network, and used a named volume for persistent MongoDB storage.

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

PS D:\Projects\DevOps Github\Assignment-5> docker compose config
name: assignment-5
services:
  mongodb:
    container_name: assignment5-mongodb
    environment:
      MONGO_INITDB_DATABASE: assignment5
    healthcheck:
      test:
        - CMD
        - mongosh
        - --quiet
        - --eval
        - db.adminCommand('ping').ok
      timeout: 5s
      interval: 5s
      retries: 10
      start_period: 10s
    image: mongo:7
    networks:
      assignment5-network: null
    ports:
      - mode: ingress
        target: 27017
        published: "27017"
        protocol: tcp
    restart: unless-stopped
    volumes:
      - type: volume
        source: mongodb-data
        target: /data/db
        volume: {}
  web:
    build:
      context: D:\Projects\DevOps Github\Assignment-5\app
      dockerfile: Dockerfile
    container_name: assignment5-web
    depends_on:
      mongodb:
        condition: service_healthy
        required: true
    environment:
      MONGO_COLLECTION: messages
      MONGO_DB: assignment5
      MONGO_URI: mongodb://mongodb:27017/assignment5
    networks:
      assignment5-network: null
    ports:
      - mode: ingress
        target: 5000
        published: "5000"
        protocol: tcp
    networks:
      assignment5-network:
        name: assignment-5_assignment5-network
        driver: bridge
    volumes:
      mongodb-data:
        name: assignment-5_mongodb-data
PS D:\Projects\DevOps Github\Assignment-5> |
```

FIGURE 1 — Compose Configuration Validation

```

#10 1.780 Collecting Jinja2==3.1.2 (from Flask==3.1.2->r requirements.txt (line 1))
#10 1.793 Downloading Jinja2-3.1.2-py3-none-any.whl.metadata (2.9 kB)
#10 1.831 Collecting MarkupSafe==2.1.1 (from Flask==3.1.2->r requirements.txt (line 1))
#10 1.845 Downloading MarkupSafe-2.1.1-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata (2.7 kB)
#10 1.868 Collecting Werkzeug==3.1.0 (from Flask==3.1.2->r requirements.txt (line 1))
#10 1.877 Downloading Werkzeug-3.1.0-py3-none-any.whl.metadata (4.0 kB)
#10 1.896 Collecting dnspython<3.0.0, >=1.16.0 (from pymongo==4.15.1->r requirements.txt (line 2))
#10 1.908 Downloading dnspython-2.8.0-py3-none-any.whl.metadata (5.7 kB)
#10 1.929 Downloading Flask-3.1.2-py3-none-any.whl (103 kB)
#10 1.956 Downloading pymongo-4.15.1-cp312-cp312-manylinux2014_x86_64.manylinux2014_x86_64.whl (1.7 MB)
#10 2.052 _____ 1.7/1.7 MB 20.2 MB/s eta 0:00:00
#10 2.063 Downloading blinker-1.9.0-py3-none-any.whl (8.5 kB)
#10 2.077 Downloading click-8.5.0-py3-none-any.whl (125 kB)
#10 2.096 Downloading dnspython-2.8.0-py3-none-any.whl (331 kB)
#10 2.126 Downloading itsdangerous-2.2.0-py3-none-any.whl (16 kB)
#10 2.141 Downloading Jinja2-3.1.2-py3-none-any.whl (134 kB)
#10 2.163 Downloading MarkupSafe-2.1.1-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (22 kB)
#10 2.177 Downloading Werkzeug-3.1.0-py3-none-any.whl (226 kB)
#10 2.229 Installing collected packages: MarkupSafe, itsdangerous, dnspython, click, blinker, Werkzeug, pymongo, Jinja2, Flask
#10 2.966 Successfully installed Flask-3.1.2 blinker-1.9.0 click-8.5.0 dnspython-2.8.0 itsdangerous-2.2.0 Jinja2-3.1.2 MarkupSafe-2.1.1 Werkzeug-3.1.0 pymongo-4.15.1
#10 2.966 WARNING: Running pip as the 'root' user can result in broken permissions and conflicting behaviour with the system package manager, possibly rendering your system unusable. It is recommended to use a virtual environment instead: https://pip.pypa.io/warnings/venv. Use the --root-user-action option if you know what you are doing and want to suppress this warning.
#10 3.036
#10 3.036 [notice] A new release of pip is available: 25.0.1 -> 26.2.1
#10 3.036 [notice] To update, run: pip install --upgrade pip
#10 DONE 3.1s

#11 [5/5] COPY app.py .
#11 DONE 0.0s

#12 exporting to image
#12 exporting layers
#12 exporting layers 0.7s done
#12 exporting manifest sha256:df31a62458484c9b3b3ddad5708091ff1f30177ffe921c3603edd090bf68cae3b done
#12 exporting config sha256:6561601ab2cb7daa935c73a71c13391068c2c46b5fb9fe32c99d3cee9c225cd3 done
#12 exporting attestation manifest sha256:931bfa958d631c26db84ccec168cb0aa628d0da012ec5f05dd840e0a97c7007 done
#12 exporting manifest list sha256:ab04fa73cfa62150b23fc2e46125bf311aldfed2abda66db9beaad1a77ce7dd done
#12 naming to docker.io/library/assignment-5-web:latest done
#12 unpacking to docker.io/library/assignment-5-web:latest
#12 unpacking to docker.io/library/assignment-5-web:latest 0.3s done
#12 DONE 1.1s

#13 resolving provenance for metadata file
#13 DONE 0.0s
[+] build 1/1
✔ Image assignment-5-web Built 7.2s
PS D:\Projects\DevOps Github\Assignment-5> |

```

FIGURE 2 — Docker Compose Build

```

PS D:\Projects\DevOps Github\Assignment-5> docker compose up -d
[+] up 4/4
✔ Network assignment-5_assignment5-network Created 0.0s
✔ Volume assignment-5_mongodb-data Created 0.0s
✔ Container assignment5-mongodb Healthy 5.8s
✔ Container assignment5-web Started 5.8s
PS D:\Projects\DevOps Github\Assignment-5> docker compose ps
NAME IMAGE COMMAND SERVICE CREATED STATUS PORTS
assignment5-mongodb mongo:7 "docker-entrypoint.s..." mongodb 10 seconds ago Up 9 seconds (healthy) 0.0.0.0:27017->27017/tcp, [::]:27017->27017/tcp
assignment5-web assignment-5-web "python app.py" web 10 seconds ago Up 4 seconds 0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp
PS D:\Projects\DevOps Github\Assignment-5> |

```

FIGURE 3 — Multi-Container Application Running

```

{
  "application": "DevOps Assignment 5",
  "database": "MongoDB",
  "endpoints": [
    "/",
    "/health",
    "/data"
  ],
  "message": "Flask web application is running with Docker Compose."
}

```

FIGURE 4 — Flask Web Application

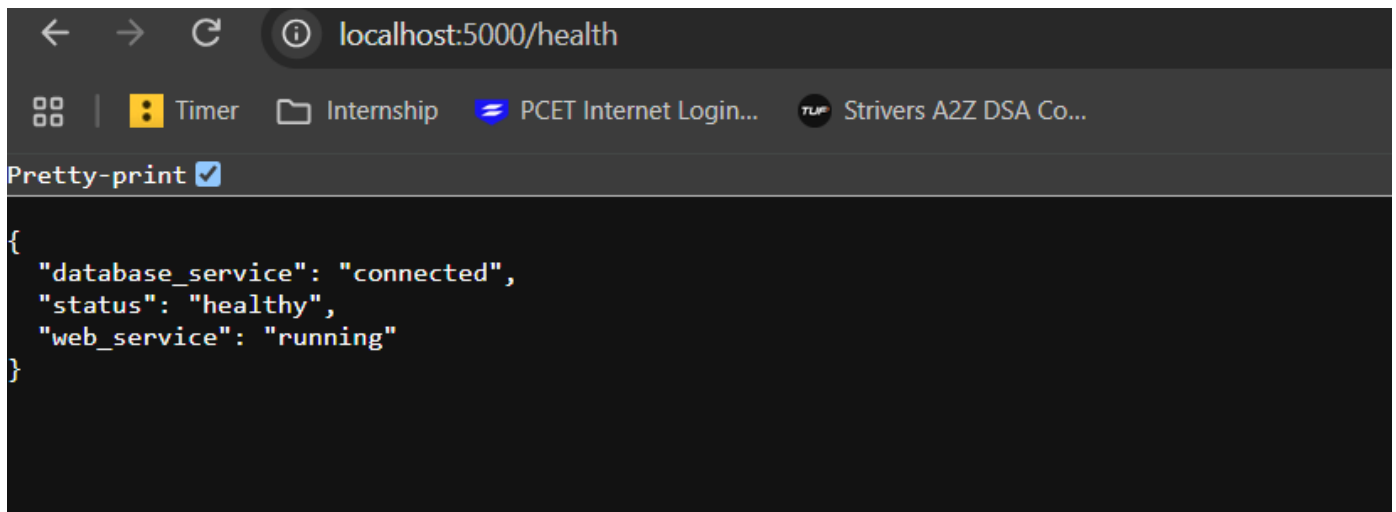


FIGURE 5 — Application and Database Health

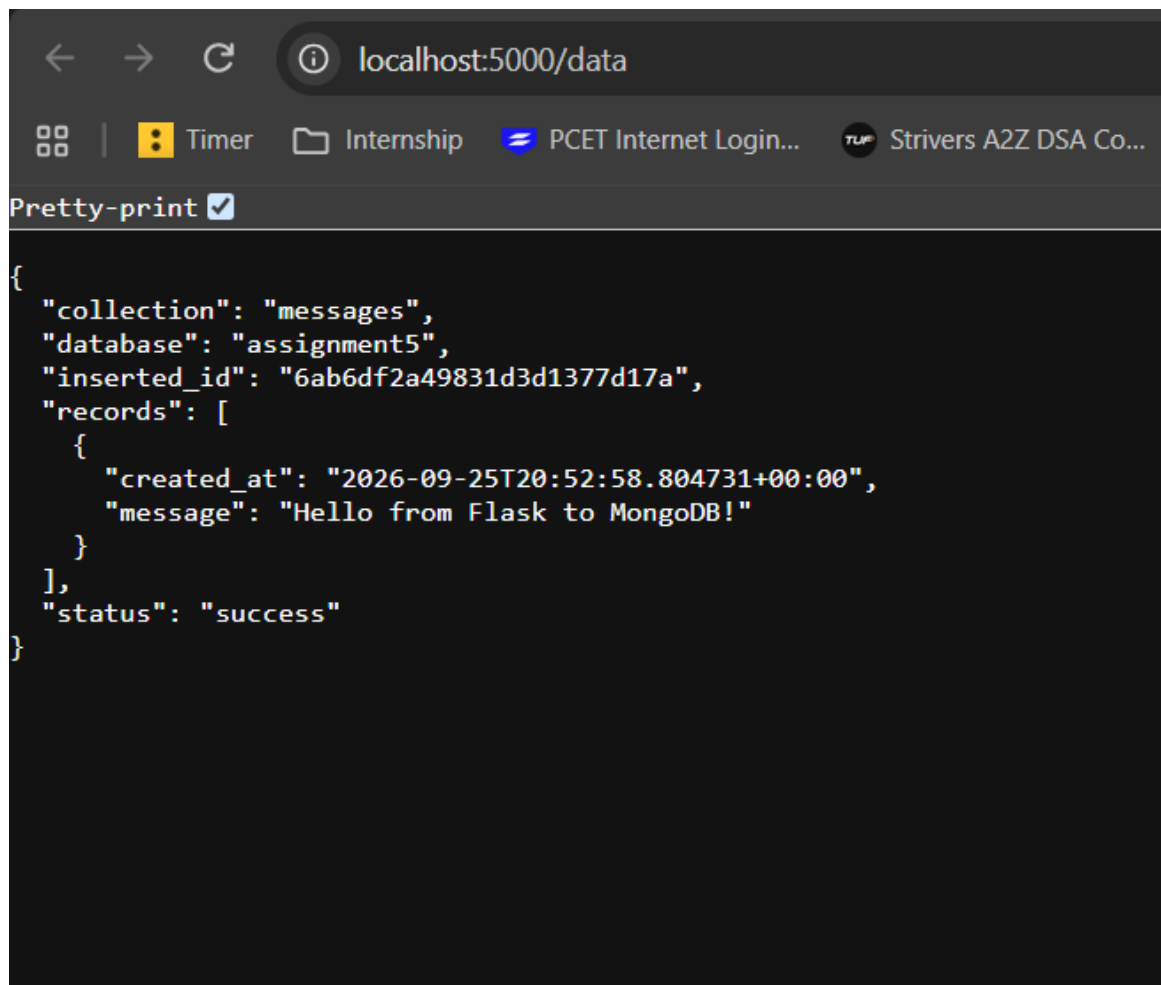


FIGURE 6 — Flask to MongoDB Communication


```

PS D:\Projects\DevOps Github\Assignment-5> docker compose logs --tail=50
assignment5-web      | * Serving Flask app 'app'
assignment5-web      | * Debug mode: off
assignment5-web      | WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
assignment5-web      | * Running on all addresses (0.0.0.0)
assignment5-web      | * Running on http://127.0.0.1:5000
assignment5-web      | * Running on http://172.18.0.3:5000
assignment5-web      | Press CTRL+C to quit
assignment5-web      | 172.18.0.1 - - [25/Sep/2026 20:50:03] "GET / HTTP/1.1" 200 -
assignment5-web      | 172.18.0.1 - - [25/Sep/2026 20:52:20] "GET /health HTTP/1.1" 200 -
assignment5-web      | 172.18.0.1 - - [25/Sep/2026 20:52:58] "GET /data HTTP/1.1" 200 -
assignment5-mongodb  | {"t":{"date":"2026-09-25T20:53:52.055+00:00"},"s":"I",  "c":"NETWORK",  "id":22944, "ctx":"conn268", "msg":"Connection ended", "attr":{"remote":"1
27.0.0.1:54666","isLoadBalanced":false,"uuid":{"uuiid":{"$uid":"71f78d3d-f32c-4f58-8f09-7b315e41be96"}}, "connectionId":268,"connectionCount":6}}
assignment5-mongodb  | {"t":{"date":"2026-09-25T20:53:52.055+00:00"},"s":"I",  "c":"NETWORK",  "id":22944, "ctx":"conn267", "msg":"Connection ended", "attr":{"remote":"1
27.0.0.1:54658","isLoadBalanced":false,"uuid":{"uuiid":{"$uid":"d26dc74-6b17-40cf-afaa-d26e424fb383"}}, "connectionId":267,"connectionCount":5}}
assignment5-mongodb  | {"t":{"date":"2026-09-25T20:53:52.056+00:00"},"s":"I",  "c":"NETWORK",  "id":22944, "ctx":"conn266", "msg":"Connection ended", "attr":{"remote":"1
27.0.0.1:54642","isLoadBalanced":false,"uuid":{"uuiid":{"$uid":"c7056ad5-d2c5-4359-a479-659b634539e2"}}, "connectionId":266,"connectionCount":4}}
assignment5-mongodb  | {"t":{"date":"2026-09-25T20:53:52.056+00:00"},"s":"I",  "c":"NETWORK",  "id":22944, "ctx":"conn265", "msg":"Connection ended", "attr":{"remote":"1
27.0.0.1:54626","isLoadBalanced":false,"uuid":{"uuiid":{"$uid":"f7dd7e9e-2a69-41f8-86ac-333e0356c0e6"}}, "connectionId":265,"connectionCount":3}}
assignment5-mongodb  | {"t":{"date":"2026-09-25T20:53:57.295+00:00"},"s":"I",  "c":"NETWORK",  "id":22943, "ctx":"listener", "msg":"Connection accepted", "attr":{"remote
":"127.0.0.1:54676","isLoadBalanced":false,"uuid":{"uuiid":{"$uid":"2042c566-b379-4872-9bd6-0a4d892c8e29"}}, "connectionId":269,"connectionCount":4}}
assignment5-mongodb  | {"t":{"date":"2026-09-25T20:53:57.296+00:00"},"s":"I",  "c":"NETWORK",  "id":51800, "ctx":"conn269", "msg":"client metadata", "attr":{"remote":"12
7.0.0.1:54676","client":"conn269", "negotiatedCompressors":[], "doc":{"application":{"name":"mongosh 2.10.0"},"driver":{"name":"nodejs|mongosh", "version":"7.5.0|2.10.0"},"p
latform":"Node.js v24.18.1, LE", "os":{"name":"linux", "architecture":"x64", "version":"3.10.0-327.22.2.el7.x86_64", "type":"Linux"},"env":{"container":{"runtime":"docker"}}}}
}}
assignment5-mongodb  | {"t":{"date":"2026-09-25T20:53:57.296+00:00"},"s":"I",  "c":"ACCESS",  "id":10483900, "ctx":"conn269", "msg":"Connection not authenticating", "attr
":{"client":"127.0.0.1:54676", "doc":{"application":{"name":"mongosh 2.10.0"},"driver":{"name":"nodejs|mongosh", "version":"7.5.0|2.10.0"},"platform":"Node.js v24.18.1, LE",
"os":{"name":"linux", "architecture":"x64", "version":"3.10.0-327.22.2.el7.x86_64", "type":"Linux"},"env":{"container":{"runtime":"docker"}}}}}
assignment5-mongodb  | {"t":{"date":"2026-09-25T20:53:57.299+00:00"},"s":"I",  "c":"NETWORK",  "id":22943, "ctx":"listener", "msg":"Connection accepted", "attr":{"remote
":"127.0.0.1:54686","isLoadBalanced":false,"uuid":{"uuiid":{"$uid":"1acb54c4-1519-4a2f-ad6a-62e74ba5beaf"}}, "connectionId":270,"connectionCount":5}}
assignment5-mongodb  | {"t":{"date":"2026-09-25T20:53:57.300+00:00"},"s":"I",  "c":"NETWORK",  "id":51800, "ctx":"conn270", "msg":"client metadata", "attr":{"remote":"12
7.0.0.1:54686","client":"conn270", "negotiatedCompressors":[], "doc":{"application":{"name":"mongosh 2.10.0"},"driver":{"name":"nodejs|mongosh", "version":"7.5.0|2.10.0"},"p
latform":"Node.js v24.18.1, LE", "os":{"name":"linux", "architecture":"x64", "version":"3.10.0-327.22.2.el7.x86_64", "type":"Linux"},"env":{"container":{"runtime":"docker"}}}}
}}
assignment5-mongodb  | {"t":{"date":"2026-09-25T20:53:57.300+00:00"},"s":"I",  "c":"ACCESS",  "id":10483900, "ctx":"conn270", "msg":"Connection not authenticating", "attr
":{"client":"127.0.0.1:54686", "doc":{"application":{"name":"mongosh 2.10.0"},"driver":{"name":"nodejs|mongosh", "version":"7.5.0|2.10.0"},"platform":"Node.js v24.18.1, LE",
"os":{"name":"linux", "architecture":"x64", "version":"3.10.0-327.22.2.el7.x86_64", "type":"Linux"},"env":{"container":{"runtime":"docker"}}}}}
assignment5-mongodb  | {"t":{"date":"2026-09-25T20:53:57.309+00:00"},"s":"I",  "c":"NETWORK",  "id":6788700, "ctx":"conn270", "msg":"Received first command on ingress con
nection since session start or auth handshake", "attr":{"elapsedMillis":9}}
assignment5-mongodb  | {"t":{"date":"2026-09-25T20:53:57.371+00:00"},"s":"I",  "c":"NETWORK",  "id":22943, "ctx":"listener", "msg":"Connection accepted", "attr":{"remote
":"127.0.0.1:54694","isLoadBalanced":false,"uuid":{"uuiid":{"$uid":"4187c705-5d7d-4fbc-aede-d60746bdcdb1"}}, "connectionId":271,"connectionCount":6}}
assignment5-mongodb  | {"t":{"date":"2026-09-25T20:53:57.371+00:00"},"s":"I",  "c":"NETWORK",  "id":22943, "ctx":"listener", "msg":"Connection accepted", "attr":{"remote
":"127.0.0.1:54700","isLoadBalanced":false,"uuid":{"uuiid":{"$uid":"903148f8-ee65-4a45-9a80-d265ec23b5fb"}}, "connectionId":272,"connectionCount":7}}
assignment5-mongodb  | {"t":{"date":"2026-09-25T20:53:57.373+00:00"},"s":"I",  "c":"NETWORK",  "id":51800, "ctx":"conn271", "msg":"client metadata", "attr":{"remote":"12
7.0.0.1:54694","client":"conn271", "negotiatedCompressors":[], "doc":{"application":{"name":"mongosh 2.10.0"},"driver":{"name":"nodejs|mongosh", "version":"7.5.0|2.10.0"},"p
latform":"Node.js v24.18.1, LE", "os":{"name":"linux", "architecture":"x64", "version":"3.10.0-327.22.2.el7.x86_64", "type":"Linux"},"env":{"container":{"runtime":"docker"}}}}
}}

```

FIGURE 7 — Docker Compose Logs

```

PS D:\Projects\DevOps Github\Assignment-5> docker network ls
NETWORK ID          NAME                                     DRIVER  SCOPE
f952fc36aee5        assignment-5_assignment5-network        bridge  local
429d0d114e4c        bridge                                  bridge  local
43364e0e4f05        host                                    host    local
95e941c8fd82        none                                    null    local

```

```

PS D:\Projects\DevOps Github\Assignment-5> docker network inspect assignment-5_assignment5-network
[
  {
    "Name": "assignment-5_assignment5-network",
    "Id": "f952fc36aee5028a6a5ceb05dc576563217ef18d4198a01d35565c1dfff907db0",
    "Created": "2026-09-25T20:49:08.827095044Z",
    "Scope": "local",
    "Driver": "bridge",
    "EnableIPv4": true,
    "EnableIPv6": false,
    "IPAM": {
      "Driver": "default",
      "Options": null,
      "Config": [
        {
          "Subnet": "172.18.0.0/16",
          "Gateway": "172.18.0.1"
        }
      ]
    },
    "Internal": false,
    "Attachable": false,
    "Ingress": false,
    "ConfigFrom": {
      "Network": ""
    },
    "ConfigOnly": false,
    "Options": {
      "com.docker.network.enable_ipv4": "true",
      "com.docker.network.enable_ipv6": "false"
    },
    "Labels": {
      "com.docker.compose.config-hash": "bda930c232a8ff611d94d88735bf5a117edc1a4bec8f66414e1f596d6ab683b2",
      "com.docker.compose.network": "assignment5-network",
      "com.docker.compose.project": "assignment-5",
      "com.docker.compose.version": "5.4.0"
    },
    "Containers": {
      "43d0e63dcdcebab610ab85628c8c23fa488c93e5f5e23d710ed3cfa5999a2e5c": {
        "Name": "assignment5-web",
        "EndpointID": "9febbacc16cc7e08cbd9b34677a1457f0a98209c5dca42944916194f96873a94",
        "MacAddress": "c6:23:41:e3:f5:c3",
        "IPv4Address": "172.18.0.3/16",
        "IPv6Address": ""
      },
      "6035dea25bf48fe8d77b150db1f585fa40ea345e1db4a99052e0a4f3575190e4": {
        "Name": "assignment5-mongodb",
        "EndpointID": "62f0c65dc6bf82499925e44fa85b7424e8759b811c81e74ee06e37f18e3d144c",
        "MacAddress": "22:fe:7b:c0:98:6a",
        "IPv4Address": "172.18.0.2/16",
        "IPv6Address": ""
      }
    },
    "Status": {
      "IPAM": {
        "Subnets": {
          "172.18.0.0/16": {
            "IPsInUse": 5,
            "DynamicIPsAvailable": 65531
          }
        }
      }
    }
  }
]
PS D:\Projects\DevOps Github\Assignment-5> |

```

FIGURE 8 — Docker Network

```

PS D:\Projects\DevOps Github\Assignment-5> docker volume ls
DRIVER          VOLUME NAME
local           6add42740c7bd6a3b8bb28b6fd0702c6409fd6ea080a828d307086d44aa74d1a
local           02269830fdff2bc05927ca3e9e2ced100e8b89f12da969b193da875baadea694
local           assignment-5_mongodb-data
local           event-management-portal_pgdata
local           fcb10da69a76727bd58bb11e5ec00eed035f8b773e598a7ee93e1b8f7f7f14f2
PS D:\Projects\DevOps Github\Assignment-5> docker volume inspect assignment-5_mongodb-data
[
  {
    "CreatedAt": "2026-09-25T20:49:08Z",
    "Driver": "local",
    "Labels": {
      "com.docker.compose.config-hash": "0a0f228d472c284ff00d50379251f84749c2e85867aabfd9dac27aa2f092303a",
      "com.docker.compose.project": "assignment-5",
      "com.docker.compose.version": "5.4.0",
      "com.docker.compose.volume": "mongodb-data"
    },
    "Mountpoint": "/var/lib/docker/volumes/assignment-5_mongodb-data/_data",
    "Name": "assignment-5_mongodb-data",
    "Options": null,
    "Scope": "local"
  }
]
PS D:\Projects\DevOps Github\Assignment-5> |

```

FIGURE 9 — MongoDB Persistent Volume

```

PS D:\Projects\DevOps Github\Assignment-5> docker compose exec mongodb mongosh
Current Mongosh Log ID: 6ab6e0e933719e38630dbdb08
Connecting to:  mongod://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.10.0
Using MongoDB:  7.0.40
Using Mongosh:  2.10.0

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

To help improve our products, anonymous usage data is collected and sent to MongoDB periodically (https://www.mongodb.com/legal/privacy-policy).
You can opt-out by running the disableTelemetry() command.

-----
The server generated these startup warnings when booting
2026-09-25T20:49:09.288+00:00: Using the XFS filesystem is strongly recommended with the WiredTiger storage engine. See http://dochub.mongodb.org/core/prodnotes-filesystem
2026-09-25T20:49:09.442+00:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
-----

test> use assignment5
switched to db assignment5
assignment5> show collections
messages
assignment5> db.messages.find().pretty()
[
  {
    _id: ObjectId('6ab6df2a49831d3d1377d17a'),
    message: 'Hello from Flask to MongoDB!',
    created_at: '2026-09-25T20:52:58.804731+00:00'
  }
]
assignment5> exit
PS D:\Projects\DevOps Github\Assignment-5> |

```

FIGURE 10 — MongoDB Data Verification

```

PS D:\Projects\DevOps Github\Assignment-5> docker compose restart mongodb
[+] restart 0/1
- Container assignment5-mongodb Restarting
PS D:\Projects\DevOps Github\Assignment-5> docker compose ps
NAME                IMAGE          COMMAND                  SERVICE    CREATED      STATUS      PORTS
assignment5-mongodb  mongo:7       "docker-entrypoint.s..." SERVICE    12 minutes ago Up 4 seconds (health: starting) 0.0.0.0:27017->27017/tcp, [::]:27017->27017/tcp
assignment5-web      assignment5-web "python app.py"         web        12 minutes ago Up 12 minutes                0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp
PS D:\Projects\DevOps Github\Assignment-5> docker compose ps
NAME                IMAGE          COMMAND                  SERVICE    CREATED      STATUS      PORTS
assignment5-mongodb  mongo:7       "docker-entrypoint.s..." SERVICE    13 minutes ago Up 24 seconds (healthy) 0.0.0.0:27017->27017/tcp, [::]:27017->27017/tcp
assignment5-web      assignment5-web "python app.py"         web        13 minutes ago Up 12 minutes                0.0.0.0:5000->5000/tcp, [::]:5000->5000/tcp
PS D:\Projects\DevOps Github\Assignment-5> docker compose exec mongodb mongosh
Current Mongosh Log ID: 6ab6e15ba50b76b90004ed98
Connecting to:  mongod://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.10.0
Using MongoDB:  7.0.40
Using Mongosh:  2.10.0

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

-----
The server generated these startup warnings when booting
2026-09-25T21:01:49.191+00:00: Using the XFS filesystem is strongly recommended with the WiredTiger storage engine. See http://dochub.mongodb.org/core/prodnotes-filesystem
2026-09-25T21:01:49.550+00:00: Access control is not enabled for the database. Read and write access to data and configuration is unrestricted
-----

test> use assignment5
switched to db assignment5
assignment5> db.messages.find().pretty()
[
  {
    _id: ObjectId('6ab6df2a49831d3d1377d17a'),
    message: 'Hello from Flask to MongoDB!',
    created_at: '2026-09-25T20:52:58.804731+00:00'
  }
]
assignment5> |

```

FIGURE 11 — Volume Persistence Verification

```

PS D:\Projects\DevOps Github\Assignment-5> docker compose down
[+] down 3/3
✓Container assignment5-web Removed
✓Container assignment5-mongodb Removed
✓Network assignment-5_assignment5-network Removed
PS D:\Projects\DevOps Github\Assignment-5> docker compose ps
NAME IMAGE COMMAND SERVICE CREATED STATUS PORTS
PS D:\Projects\DevOps Github\Assignment-5> docker volume ls
DRIVER VOLUME NAME
local 6add42740c7bd6a3b8bb28b6fd0702c6409fd6ea080a828d307086d44aa74d1a
local 02269830fdff2bc05927ca3e9e2ced100e8b89f12da969b193da875baadea694
local assignment-5_mongodb-data
local event-management-portal_pgdata
local fcb10da69a76727bd58bb11e5ec00eed035f8b773e598a7ee93e1b8f7f7f14f2
PS D:\Projects\DevOps Github\Assignment-5> |

```

FIGURE 12 — Docker Shutdown

Result:

The multi-container application was successfully created using Docker Compose. The Flask application communicated with MongoDB through the Compose network, environment variables were used for database configuration, and MongoDB data persisted through a named Docker volume. The application was successfully stopped and cleaned up using Docker Compose.

Conclusion:

This assignment demonstrated the practical use of Docker Compose for orchestrating a multi-container application. It covered service configuration, port mapping, environment variables, health checks, networking, persistent storage, container lifecycle management, and communication between a web application and database service.

References:

1. Docker Documentation — Docker Compose
2. Docker Documentation — Volumes and Networking
3. MongoDB Documentation — MongoDB Docker Image
4. Flask Documentation
5. PyMongo Documentation

Submission Links:

GitHub Repository Link: <https://github.com/SwayamMandhani06/DevOps/tree/main/Assignment-5>

Hosted / Output Link: NA – Docker compose based local practical